Taller de SQL

202501 - ECON 1306

Contenido

- **Joins**
  - Producto cartesiano
  - Inner join
  - Outer join
  - Left join
  - Right join

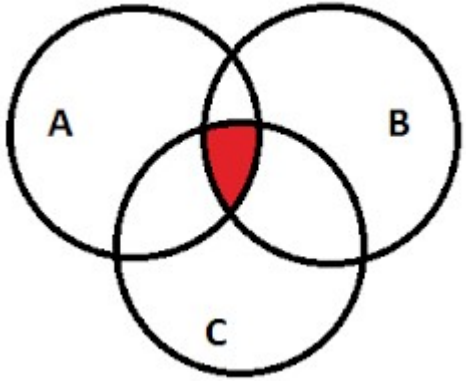
- **Unión e intersección de tablas**

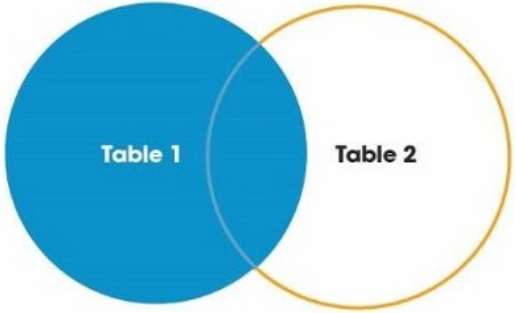
JOINS

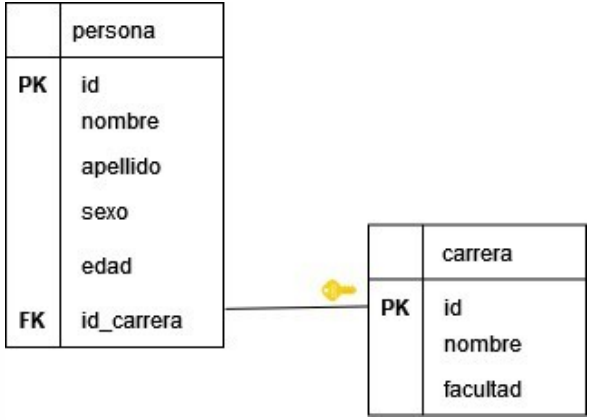
JOINS

- Los Joins son operaciones utilizadas en SQL para combinar datos de dos o más tablas en función de una relación específica entre ellas.

- Se utilizan para recuperar información relevante al combinar





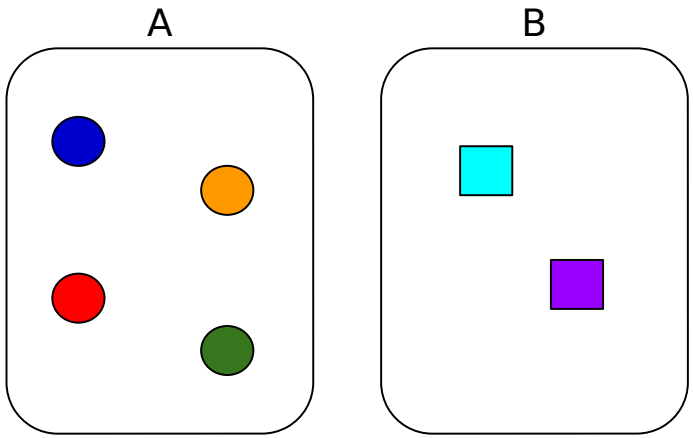


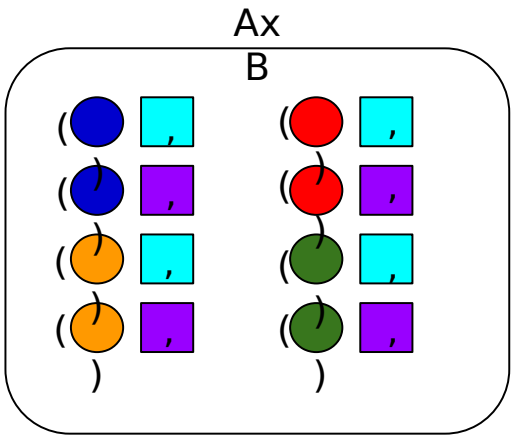
Producto cartesiano

- Es la aproximación más intuitiva de combinar datos de distintas tablas.

- **Producto cartesiano:**

  Si tengo dos conjuntos A y B, el producto cartesiano AxB serán tuplas (a,b) para elementos a en A y b en B.





Producto cartesiano

- Es la aproximación más intuitiva de combinar datos de distintas tablas.

- **Producto cartesiano:**

  Si tengo dos conjuntos A y B, el producto cartesiano AxB serán tuplas (a,b) para elementos a en A y b en B.

- **¿Si A tiene N elementos y B tiene M elementos, cuántos elementos tiene AxB?**

$$N \times M$$

Producto cartesiano

persona

id	nombre	apellido	edad	id_carrera
1	Melissa	Robles	25	0
2	Daniel			1

carrera

id	nombre	facultad
0	Matemáticas	Ciencias
1	Economía	Economía

persona x carrera

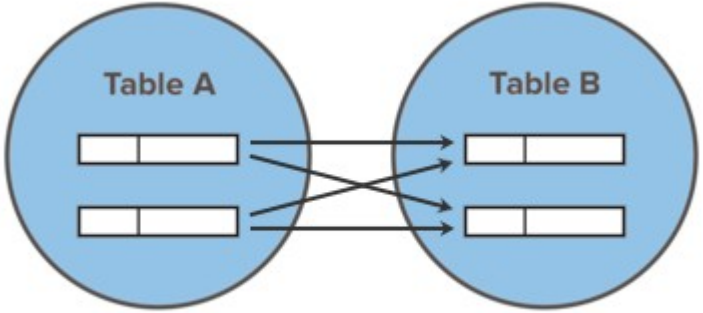
id	nombre	apellido	edad	id_carrera	id	nombre	facultad
1	Melissa	Robles	25	0	0	Matemáticas	Ciencias
1	Melissa	Robles	25	0	1	Economía	Economía
2	Daniel	Aguirre		1	0	Matemáticas	Ciencias
2	Daniel	Aguirre		1	1	Economía	Economía

Producto cartesiano

CÓDIGO: Dos opciones

```
SELECT *
FROM persona, carrera
```

```
SELECT *
FROM persona CROSS JOIN
carrera
```



SQL CROSS JOIN

¿Hay algún problema con el producto cartesiano?

Inner Join

Si Melissa estudia matemáticas no me interesa la información de la carrera de economía.

- En el CROSS JOIN la mayoría de los registros no son muy útiles.

persona x carrera

id	nombre	apellido	edad	id_carrera	id	nombre	facultad
1	Melissa	Robles	25	0	0	Matemáticas	Ciencias
1	Melissa	Robles	25	0	1	Economía	Economía

- Los otros tipos de JOIN intentan quitar estas redundancias. El principal es INNER JOIN.
- INNER JOIN pone una **condición** sobre el CROSS JOIN:

  Solo me interesa pegar los registros que tienen sentido, es decir de la carrera que debe ser

persona.id_carrera = carrera.id

Inner Join

CÓDIGO: Idea

```
SELECT *
FROM persona CROSS JOIN
carrera
WHERE persona.id_carrera =
carrera.id
```

CÓDIGO

```
SELECT *
FROM persona
INNER JOIN carrera
        ON  persona.id_carrera =
carrera.id
```

persona INNER JOIN carrera

id	nombre	apellido	edad	id_carrera	id	nombre	facultad
1	Melissa	Robles	25	0	0	Matemáticas	Ciencias
2	Daniel	Aguirre		1	1	Economía	Economía

Inner Join

Pregunta:

¿Si la tabla A tiene N elementos y la tabla B tiene M elementos, cuántos elementos tiene A INNER JOIN B?

a) NxM

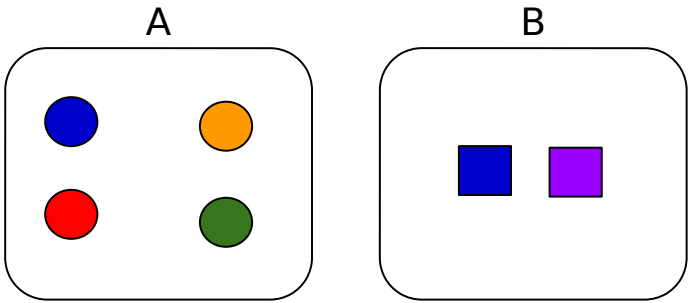
b) N

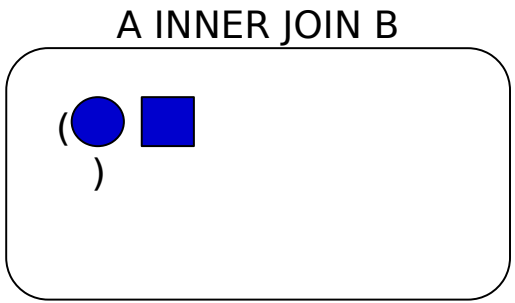
c) M

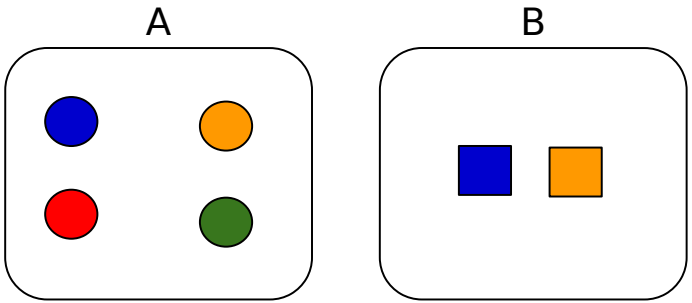
d) Otro

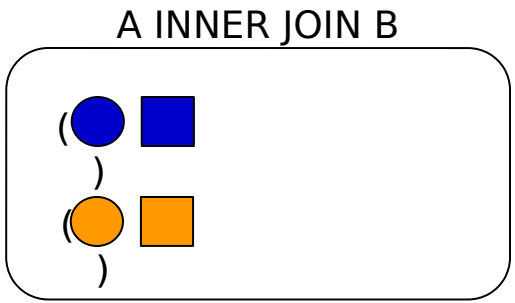
Inner Join

Condición: A.color = B.color



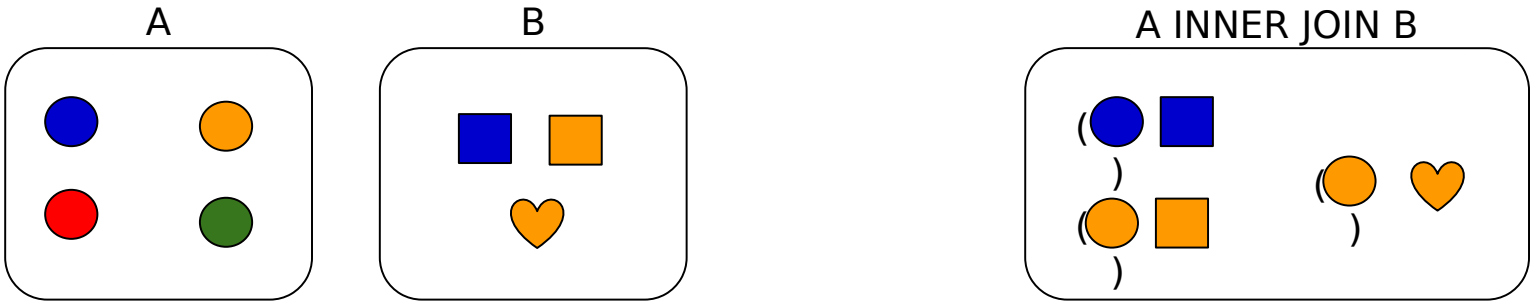






Inner Join

Condición: A.color = B.color



A diferencia del producto cartesiano, no se puede saber con certeza el número de datos de la tabla resultante. Lo único que sabemos es que tiene **menos de NxM elementos**.

Para garantizar que tenga un número menor a N y a M, la condición se debe aplicar sobre una **variable que no se repita** (por ejemplo, una llave primaria).

Inner Join

¿Sobre qué variables se realiza la condición?

- La condición se puede hacer sobre cualquier par de columnas que tengan el **mismo tipo de dato**.

- Aún así, se recomienda hacerlo sobre llaves primarias, llaves foráneas o columnas únicas en la tabla (ej. Correo electrónico).

- **Importante:** Los valores nulos nunca cumplen la condición

Tabla A

col1	col2
A1	1
A2	NULL

Tabla B

var1	var2
B1	1

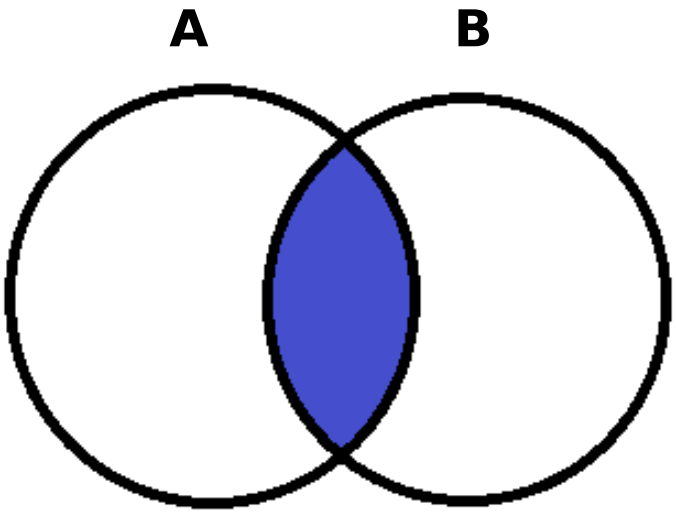
```
SELECT *
FROM A INNER
JOIN B ON A.col2 =
B.var2
```

col1	col2	var1	var2
A1	1	B1	1

* A2 no aparece en la tabla resultante

Inner Join

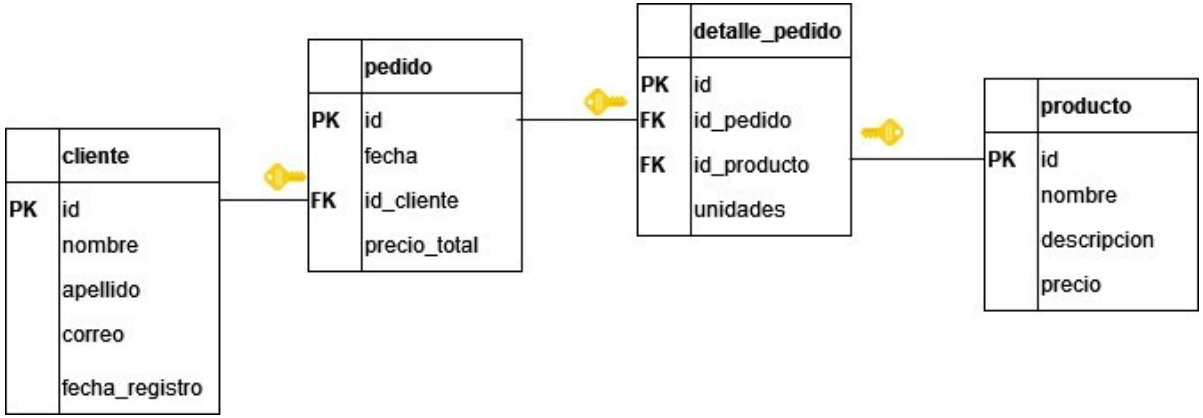
Representación de conjuntos:



Los valores resultado del **INNER JOIN** tienen que ser valores que existen en **A** y en **B** al mismo tiempo (intersección).

Inner Join

Múltiples tablas




```
SELECT *
FROM cliente AS c
INNER JOIN pedido AS p ON (c.id = p.id_cliente)
INNER JOIN detalle_pedido AS dp ON (dp.id_pedido =
p.id)
INNER JOIN producto AS pr ON (pr.id = dp.id_producto)
```

Si tengo muchas columnas con el mismo nombre, mi tabla resultante va a tener columnas que se llaman igual.

Es importante utilizar el **AS** para darle nombres específicos a las columnas resultantes.

¿Cuántas columnas tendrá el resultado de

Natural Join

- Se utiliza cuando hay una columna en común entre las dos tablas.
- SQL intuye cuál es la condición sobre la que se realiza el JOIN.
- No es necesario indicar las columnas que entrarán en la condición.

- **Nota**: Es mejor siempre identificar a mano las columnas que se van a utilizar en la condición. Es preferible usar INNER JOIN que NATURAL JOIN si no son muy claros los nombres de las columnas.

Tabla A

col1	var2
A1	1
A2	NULL

Tabla B

var1	var2
B1	1

```
SELECT *
FROM A NATURAL JOIN B
```

col1	var1	var2
A1	B1	1

Full Outer Join

- Se utiliza cuando quiero que los valores nulos se mantengan.

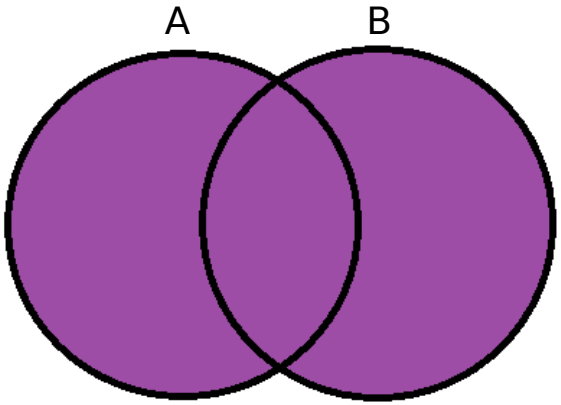


Tabla A

col1	col2
A1	1
A2	NULL

Tabla B

var1	var2
B1	1
B2	NULL

Tabla A **OUTER JOIN** Tabla B
ON A.col2 = B.var2

col1	col2	var1	var2
A1	1	B1	1
A2	NULL	NULL	NULL
NULL	NULL	B2	NULL

Left Join

- Se mantienen únicamente los valores que existen en la **primera tabla.**

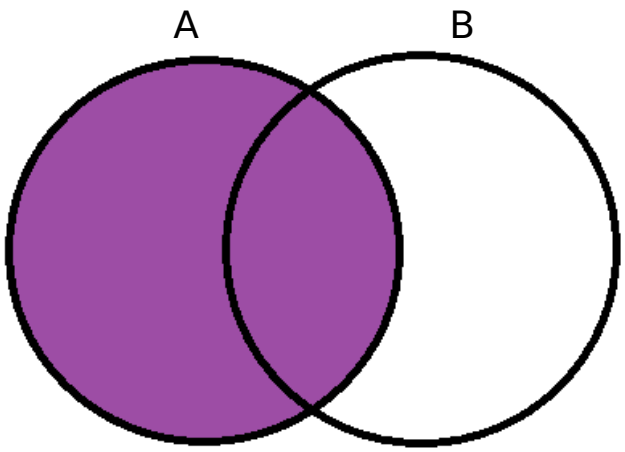


Tabla A

col1	col2
A1	1
A2	NULL

Tabla B

var1	var2
B1	1
B2	NULL

Tabla A **LEFT JOIN** Tabla B
ON A.col2 = B.var2

col1	col2	var1	var2
A1	1	B1	1
A2	NULL	NULL	NULL

Right Join

¿Has oído hablar del 'RIGHT JOIN' en SQL? Es como el amigo al que siempre invitas a tus fiestas pero que nunca viene. Puede que exista, pero rara vez lo necesitas.

- Se mantienen únicamente los valores que existen en la **segunda tabla.**

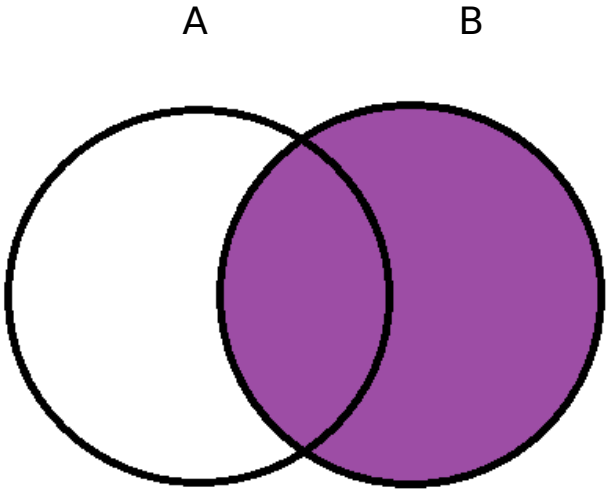


Tabla A

col1	col2
A1	1
A2	NULL

Tabla B

var1	var2
B1	1
B2	NULL

Tabla A **RIGHT JOIN** Tabla B
ON A.col2 = B.var2

col1	col2	var1	var2
A1	1	B1	1
NULL	NULL	B2	NULL

Outer Joins

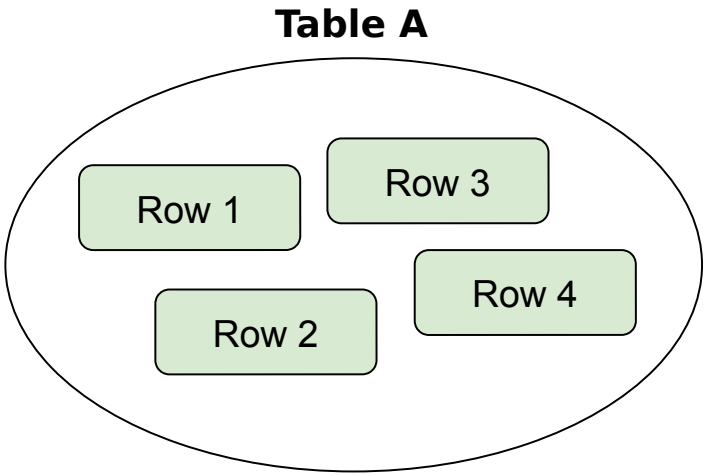
¿El orden importa?

- ☒ **INNER JOIN**
- ☒ **FULL OUTER JOIN**
- ☑ **LEFT JOIN**
- ☑ **RIGHT JOIN**

UNIONES E INTERSECCIONES

Operaciones de conjuntos

- Las filas de una tabla son únicas, por lo que las tablas pueden ser vistas como conjuntos



Por lo tanto, se pueden hacer operaciones de conjuntos sobre las tabla.

Importante: Para hacer operaciones de conjuntos entre dos tablas A y B, estas deben cumplir:

1. Tener el mismo número de columnas.
2. Tener el mismo nombre de columnas.
3. Las columnas deben tener el mismo tipo de dato.

Operaciones de conjuntos

Table 7-1. *Four Set Operations and Their Symbols*

Operation	Symbol	SQL Keyword
Union	∪	UNION
Intersection	∩	INTERSECT
Difference	–	EXCEPT

UNION

personaA

nombre	apellido	edad
Melissa	Robles	25
Daniel	Aguirre	23

personaB

nombre	apellido	edad
Melissa	Robles	25
Laura	Carmona	25

personaA UNION personaB

nombre	apellido	edad
Melissa	Robles	25
Daniel	Aguirre	23
Laura	Carmona	25

Aunque el dato Melissa Robles 25 aparece en ambas tablas, como no pueden haber registros repetidos solo aparece una vez

INTERSECCION

personaA

nombre	apellido	edad
Melissa	Robles	25
Daniel	Aguirre	23

personaB

nombre	apellido	edad
Melissa	Robles	25
Laura	Carmona	25

personaA INTERSECT personaB

nombre	apellido	edad
Melissa	Robles	25

El dato Melissa Robles 25 es el único que aparece en ambas tablas

EXCEPT

personaA

nombre	apellido	edad
Melissa	Robles	25
Daniel	Aguirre	23

personaB

nombre	apellido	edad
Melissa	Robles	25
Laura	Carmona	25

personaA EXCEPT personaB

nombre	apellido	edad
Daniel	Aguirre	23

El dato Daniel Aguirre 23 es el único que aparece en la primera tabla pero no en la segunda

EJERCICIOS

Ejercicio

En MySQL **no existe** el FULL OUTER JOIN

¿Cómo podemos replicarlo a partir de lo que aprendimos hoy?

```
SELECT *
FROM A LEFT JOIN B
ON A.col2 = B.var2
```

col1	col2	var1	var2
A1	1	B1	1
A2	NULL	NULL	NULL

```
SELECT *
FROM A RIGHT JOIN B
ON A.col2 = B.var2
```

col1	col2	var1	var2
A1	1	B1	1
NULL	NULL	B2	NULL

UNION

Tabla A

col1	col2
A1	1
A2	NULL

Tabla B

var1	var2
B1	1
B2	NULL

Ejercicio

En MySQL **no existe** el FULL OUTER JOIN

¿Cómo podemos replicarlo a partir de lo que aprendimos hoy?

```
(
    SELECT *
    FROM A LEFT JOIN B
    ON A.col2 = B.var2
)
UNION
(
    SELECT *
    FROM A RIGHT JOIN B
    ON A.col2 = B.var2
)
```

Tabla A

col1	col2
A1	1
A2	NULL

Tabla B

var1	var2
B1	1
B2	NULL

Ejercicio

Para muchas empresas es importante detectar los clientes que se registraron, realizaron pedidos y después no lo volvieron a hacer.

Con esta arquitectura, encuentra el nombre de los clientes que realizaron pedidos antes de 2023-03-01, pero que no realizaron pedidos después de esa fecha.

cliente

id	nombre
1	Melissa
2	Daniel
3	Laura

pedido

id	cliente_id	fecha
1	1	2023-01-15
2	2	2023-02-20
3	1	2023-03-10
4	3	2023-04-05

Ejercicio

Para muchas empresas es importante detectar los clientes que se registraron, realizaron pedidos y después no lo volvieron a hacer.

Con esta arquitectura, encuentra el nombre de los clientes que realizaron pedidos antes de 2023-03-01, pero que no realizaron pedidos después de esa fecha.

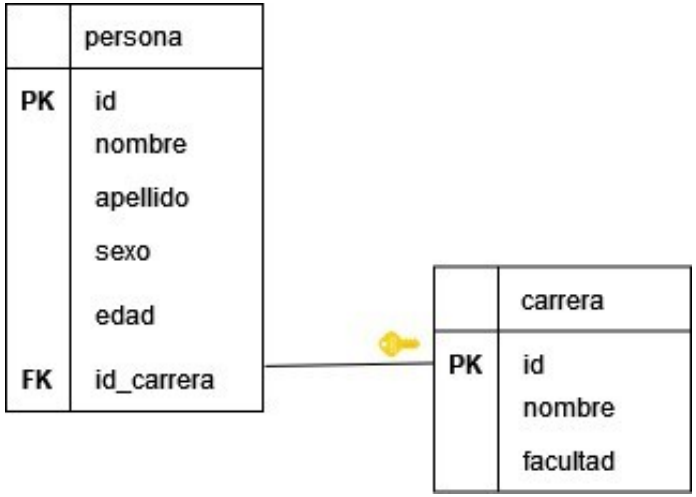
```
(
    SELECT DISTINCT(c.id)
    FROM cliente AS c
            INNER JOIN pedido AS p ON c.id =
p.id_cliente
    WHERE p.fecha < “2023-03-01”
)
EXCEPT
(
    SELECT DISTINCT(c.id)
    FROM cliente AS c
            INNER JOIN pedido AS p ON c.id =
p.id_cliente
    WHERE p.fecha > “2023-03-01”
)
```

Ejercicio

La universidad quiere conocer el número de estudiantes por facultad.

¿Con esta arquitectura, cómo se podría responder a la pregunta?

Tenga en cuenta que pueden haber carreras sin estudiantes.



Ejercicio

La universidad quiere conocer el número de estudiantes por facultad.

¿Con esta arquitectura, cómo se podría responder a la pregunta?

Tenga en cuenta que pueden haber carreras sin estudiantes.

```
SELECT c.facultad, COUNT(c.id) AS count
FROM carrera AS c
LEFT JOIN persona AS p ON c.id = p.id_carrera
GROUP BY c.facultad
```